

Smart Resume Screener & ATS Feedback Portal

1. PROBLEM STATEMENT

For Job Seekers: Applicants face a black-box screening process without visibility into why their resumes fail automated Applicant Tracking Systems (ATS). They lack actionable feedback regarding missing technical competencies, keyword optimizations, and tools to measure resume performance against targeted job descriptions.

For Recruiters: Hiring teams manually screen hundreds of resumes per job opening—a process that is slow, inefficient, and prone to human bias. Traditional keyword search filters evaluate words in isolation without measuring contextual skill relevance or providing visual tools for side-by-side applicant analytics.

2. PROPOSED SYSTEM

An intelligent dual-sided recruitment portal that automates resume text parsing, evaluates contextual and mathematical candidate compatibility, and delivers visual analytics dashboards tailored for both job seekers and hiring teams.

Core Microservice Components

- **Java Module:** Employs basic Object-Oriented Programming (OOP) classes and Apache PDFBox to extract raw text from PDF resume batches.
- **Python FastAPI Service:** Runs ML vector algorithms and mathematical scoring models to compute match percentages and skill gaps.

Data Persistence & Frontend UI

- **PlanetScale (MySQL):** Cloud database storing candidate profiles, job descriptions, parsed skill arrays, and score histories.
- **Web Dashboard:** Built with HTML5, Native CSS3, ES6+ JavaScript, and Chart.js (Score Gauges, Radar Charts, Bar Graphs).

3. SYSTEM OBJECTIVES

- **Automated PDF Parsing:** Parse unstructured PDF resume text using structured Java OOP classes and Apache PDFBox.
- **Semantic & Contextual Matching:** Calculate contextual relevance using TF-IDF Vectorization and Cosine Similarity in Python.
- **Mathematical Skill Scoring:** Evaluate hard skill overlap using the Jaccard Similarity Index, keyword frequency scaling, and weighted averages.
- **Candidate & Recruiter Dashboards:** Provide job seekers with missing skill feedback and offer recruiters interactive candidate ranking visuals via Chart.js.
- **Cloud Data Persistence:** Persist candidate records and match analytics securely in PlanetScale (MySQL).

4. SCOPE

In Scope

- PDF text extraction via Java OOP classes.
- TF-IDF & Cosine Similarity semantic matching.
- Jaccard Index skill overlap & weighted scoring.
- Candidate feedback & recruiter comparative charts.
- PlanetScale storage & Docker deployment on Render.

Out of Scope

- Optical Character Recognition (OCR) for scanned PDFs.
- Automated candidate emailing or interview scheduling.
- Multi-tenant enterprise role-based authorization (RBAC).

5. TECHNOLOGY STACK (6 CORE SUBJECTS)

SUBJECT AREA	TECHNOLOGIES & CONCEPTS APPLIED
Subject 1: Machine Learning	Scikit-Learn, TF-IDF Vectorizer, Cosine Similarity
Subject 2: Applied Math & Statistics	Jaccard Similarity Index, Min-Max Normalization, Term Frequency, Weighted Averages
Subject 3: Full-Stack Development	FastAPI (Python), HTML5, Native CSS3, Vanilla JavaScript (ES6+), PlanetScale (MySQL)
Subject 4: Software Engineering	RESTful APIs, Layered Architecture, Docker, Render, Git Repository Strategy
Subject 5: Data Visualization	Chart.js (Radar Charts, Doughnut Score Gauges, Comparison Bar Graphs)
Subject 6: Java & OOP	Java 17, Basic OOP (Classes, Inheritance, Polymorphism), Apache PDFBox

6. EXPECTED OUTCOMES & SCORING FORMULA

The system outputs transparent, deterministic candidate match ratings (\$0-100\%\$) combining semantic context, direct skill overlap, and normalized keyword density:

$$\text{Final Score} = (w_1 \times \text{Cosine Score}) + (w_2 \times \text{Jaccard Index}) + (w_3 \times \text{Normalized Frequency})$$

- **Job Seeker Portal:** Actionable missing skill feedback, resume improvement suggestions, and score evolution tracking across resume versions.
- **Recruiter Portal:** Automated candidate ranking leaderboards, side-by-side skill comparison bar graphs, and radar gap analyses for fast shortlisting.
- **Production Infrastructure:** Fully containerized Docker setup deployed seamlessly on Render cloud hosting.

7. PROJECT TIMELINE (13 WEEKS)

Timeline	Phase Deliverable	Core Activities
Weeks 1–2	Setup & Schema Design	Configure project repositories and provision PlanetScale MySQL schema.
Weeks 3–4	Java Processing Engine	Develop Java OOP parser classes using Apache PDFBox for text extraction.
Weeks 5–7	ML & Scoring Engine	Build Python TF-IDF, Cosine Similarity, and Jaccard scoring formulas.
Weeks 8–9	API & Software Eng.	Develop FastAPI REST endpoints, layered architecture, and Docker setup.
Weeks 10–11	Web UI & Analytics	Construct HTML/CSS/JS frontend and integrate Chart.js widgets.
Weeks 12–13	Deployment & Testing	Deploy on Render, conduct end-to-end integration testing, and finalize report.

Student Name: Hemant Tyagi

Roll No: 22515500039

Section: NE (L1)